

Hack the North 2026 Frontend Development Challenge

Part 2: Writeup

Nikhil Doal

1. Walk us through your development process as you worked on this project. How did you plan out the structure and design of it? How did you decide on the tools you've used? Did you encounter any problems? And if so, how did you solve them? Are there any areas of your code that you're particularly proud of or want to point out?

When I began this project, I started by carefully reviewing and mapping out the project requirements. From this, I identified three core user experiences that would shape the application:

- 1.) Discovering and exploring event information
- 2.) Navigating schedules and sessions
- 3.) Logging in to a hacker account to view personal information and participate

With this in mind, I sketched some small architecture and page structure before writing any code. This allowed me to clearly define routes, layout structure, and component responsibilities early on, which made the implementation process smoother and more organized.

Since I had prior experience with Next.js, I chose to build the project using Next.js with TypeScript and Tailwind CSS. This stack provided an excellent balance of performance, scalability, and development speed. I also carefully reviewed the provided API and structured my components around the data models it exposed, which ensured that the UI aligned naturally with backend capabilities. Throughout development, I prioritized modular design, breaking the UI into small, reusable components to keep the codebase organized, readable, and extensible.

Because of time constraints, I used GitHub Copilot and Claude Code to accelerate repetitive development tasks and explore implementation ideas. Rather than relying on them blindly, I treated them as productivity tools, reviewing, refactoring, and adapting generated code to ensure clarity, correctness, and maintainability.

From a design perspective, I aimed to create a cohesive visual theme inspired by origami and crafting, which fits naturally with Hack the North's branding. This included subtle animations such as a paper-plane transition and soft motion effects. I initially planned to implement a complex paper-folding animation for event cards, but after experimenting, I decided that the performance cost and implementation complexity outweighed the visual benefit. I instead focused on clean interactions, smooth transitions, and responsiveness across device sizes.

Summary

Tools & technologies

- **Next.js + React + TypeScript:** routing, performance, scalability, and type safety.
- **Tailwind CSS:** fast iteration and consistent styling.
- **Framer Motion & animation utilities:** smooth UI transitions.

Challenges & how I solved them

- **Maintaining visual and structural consistency:** solved by extracting reusable layout and UI components.
- **Balancing design ambition vs. performance and feasibility:** removed overly complex animations in favor of smoother, simpler interactions.
- **Ensuring responsiveness across screen sizes:** designed mobile-first layouts and tested across breakpoints.

2. Given additional time, how would you extend your application to become a fully functional product that thousands of hackers and the general public would use at Hackathon Global Inc.TM's next event? Would you add more features and performance metrics? If so, what would they be?

With additional time, I would extend this application into a fully production-ready hackathon platform capable of supporting thousands of concurrent users.

More features

- Full authentication and user dashboard system
- Personalized hacker profiles
- Live schedule updates and reminders
- Team formation and collaboration tools
- Advanced event filtering and recommendations

Analytics & performance metrics

- User engagement tracking (clicks, navigation paths, feature usage)
- Registration funnel analysis
- Performance monitoring (page load times, interaction latency)
- Real-time usage metrics during event peaks

Scalability improvements

- Edge caching and server-side rendering optimizations
- Load balancing and API optimization
- Improved error handling and logging pipelines

I would also revisit advanced UI ideas such as dynamic card animations and richer visual transitions once performance and accessibility guarantees are fully met.

3. Any other thoughts you have (not limited to the previous questions)

I fully implemented all the required functionality and extended the project with additional features, including persistent event favoriting (saved across refreshes) and advanced filtering based on event type. These improvements enhance usability and provide a more personalized experience.

Throughout development, I focused on ensuring that:

- The codebase is readable and understandable, even for developers unfamiliar with the project.
- The structure is modular and extensible, making it easy to add new event types or pages.
- Best practices are followed to maintain long-term scalability.
- The application is responsive across devices, supporting both desktop and mobile experiences.
- The design remains visually cohesive and engaging, reinforced through a consistent origami-inspired theme.

If this project were to continue long-term, I would focus on further accessibility improvements, performance profiling, and usability testing to ensure the platform meets the needs of a diverse and large user base.